A6 Data quality

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track A: Databricks pipeline engineering 2
A6 Data quality 3
A6-L1 Data contracts with severity levels and structured results 3
A6-L2 Statistical checks that survive empty, single-row and zero data 8
A6-L3 Reconcile counts and totals between layers 13
Track A best-practice checklist 18
Appendix: solutions and answer keys 19

Track A: Databricks pipeline engineering

Build Databricks pipelines that are correct, fast, observable and tested.

A6 Data quality

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Expectations, health checks and reconciliation between layers.

A6-L1 Data contracts with severity levels and structured results

Concept

A **data contract** states what a delivery must look like before it may enter silver: row count range, mandatory columns, weight total, unique keys, allowed categories, expected distributions. Your reference project keeps each contract as versioned JSON, such as `nat_contract_v2.json`.

Every check produces a **structured result**, never a sentence:

```python
CheckName = str
Status = Literal["PASS", "WARN", "FAIL", "SKIP"]
Severity = Literal["CRITICAL", "HIGH", "MEDIUM", "LOW"]

@dataclass(frozen=True)
class CheckResult:
    check_name: CheckName
    status: Status
    severity: Severity
    expected: str
    actual: str
```

Records can be counted, stored in a Delta table, charted and compared across waves. A narrative such as "weights look a bit high" cannot.

Severity decides what happens next.

- A failed **CRITICAL** check stops the pipeline: raise an exception, so the job fails and silver stays untouched. Row count out of range, null keys, duplicate keys and a weight total outside limits are CRITICAL.
- A **WARN** logs the result and the run continues. Unexpected category values and distribution shifts are warnings: worth a look, not worth a missed report.

Evaluate every check first, store all the results, then decide. If you raise on the first failure, you learn about one problem per run.

Keep the contract in data and the logic in code. A new contract version changes JSON, not Python.

On Databricks, Lakeflow Spark Declarative Pipelines offer the same idea declaratively: `@dp.expect` records violations, `@dp.expect_or_drop` removes violating rows, and `@dp.expect_or_fail` stops the update. Use them when your pipeline runs in that framework. The contract pattern here works in any job.

Why it matters

A contract validator raises on the first failed check. A redelivery has a row count 2% below the minimum, so the run stops at check 1. The supplier fixes the count and resends. The next run stops at check 4, duplicate keys. The third run stops at check 6, a weight total outside the range. Three round trips with the supplier cost four days.

Evaluating every check, storing all results, and then raising once for all CRITICAL failures gives the supplier one complete list on the first day.

Worked example: weak code

```
1  def validate(df, contract):
2      if df.count() < contract["row_count_min"]:
(1)        raise Exception("Too few rows")
4      if df.filter("respondent_id is null").count() > 0:
(2)        raise Exception("Null ids found")
6      if df.select("gender").distinct().count() > 4:
(3)        print("warning: odd genders")
(4)    return "ok"
```

1. **Line 3.** Stops at the first failure, with no numbers. Later checks never run.
2. **Line 5.** A bare Exception with a sentence: nothing to store, count or compare.
3. **Line 7.** A warning that goes to stdout and is lost.
4. **Line 8.** The caller learns nothing about which checks ran.

Worked example: strong code

```
1  from dataclasses import dataclass
2  from typing import Literal
3
(1) Status = Literal["PASS", "WARN", "FAIL", "SKIP"]
5  Severity = Literal["CRITICAL", "HIGH", "MEDIUM", "LOW"]
6
7
8  @dataclass(frozen=True)
(2) class CheckResult:
10     """One contract check outcome, ready to store in the QA results
       table."""
11
12     check_name: str
13     status: Status
```

```
    severity: Severity
    expected: str
    actual: str


class ContractViolation(Exception):
    """Raised when one or more CRITICAL checks fail."""


def critical_failures(results: list[CheckResult]) -> list[CheckResult]:
    """Failed checks with CRITICAL severity."""
    return [r for r in results if r.status == "FAIL" and r.severity == "CRITICAL"]


def enforce(results: list[CheckResult]) -> None:
    """Raise once, listing every CRITICAL failure. WARN results pass through."""
    failures = critical_failures(results)
    if failures:
        names = ", ".join(f"{r.check_name} (expected {r.expected}, got {r.actual})" for r in failures)
        raise ContractViolation(f"{len(failures)} critical check(s) failed: {names}")
```

1. **Line 4.** Typed statuses: a typo such as "FAILED" is a mypy error.
2. **Line 9.** A record with expected and actual values, stored for every check, pass or fail.
3. **Line 19.** A specific exception type the orchestrator can catch, log and re-raise.
4. **Line 28.** Decides after every check has run, and reports all CRITICAL failures at once.

Results are data, severity decides the action, and one run reports every problem.

Recap card

- **What does a contract check produce?** A structured record: check_name, status, severity, expected, actual. Never a sentence.
- **CRITICAL versus WARN** CRITICAL failures stop the pipeline by raising. WARN results are logged and the run continues.
- **Why evaluate every check before raising?** One run reports every problem, so the supplier fixes everything in one round trip.
- **Lakeflow Declarative Pipelines expectations** @dp.expect keeps and records, @dp.expect_or_drop drops, @dp.expect_or_fail stops the update.

Practice

Exercise 1. Architecture design (level 1)

Assign a severity to each contract check.

Options: CRITICAL: stop the pipeline, WARN: log and continue.

1. Duplicate respondent_id values within a wave
2. Weight total outside the contracted range
3. 12 rows with an unexpected gender label
4. Share of 16-34s at 19%, just below the expected 20-35%

Exercise 2. Multiple choice (level 2)

A validator finds three problems in a delivery: a low row count (CRITICAL), null keys (CRITICAL) and odd labels (WARN). What should one run report?

- a) Only the first CRITICAL failure, then stop.
- b) All three results stored, then one exception naming both CRITICAL failures.
- c) Nothing: warnings and failures are only printed.
- d) All three results stored, and the run continues to silver.

Why?

- r1) Evaluate everything, record everything, then act on severity once.
- r2) Exceptions should always be raised as early as possible.

Exercise 3. Fill the gap (level 2)

Type the result fields so mypy rejects any status or severity outside the allowed values.

```
Status = _____(1)["PASS", "WARN", "FAIL", "SKIP"]


@dataclass(_____(2)=True)
class CheckResult:
    check_name: str
    status: Status
```

Exercise 4. Write the function (level 4)

Results arrive as dicts with keys `check_name`, `status`, `severity`, `expected`, `actual`. Write `enforce(results)`:

- Return the list of WARN results.

- Raise `ContractViolation` (define it, subclassing `Exception`) when any result has status `FAIL` and severity `CRITICAL`. The message must name every such check.
- A FAIL with a non-CRITICAL severity does not raise.

Your code must pass these checks:

- With no CRITICAL failure, return the WARN results.
- Raise ContractViolation naming every CRITICAL failure, not only the first.
- A FAIL with MEDIUM severity must not raise.
- ContractViolation must be its own subclass of Exception.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Raised when one or more CRITICAL checks fail.
class ContractViolation(Exception): ...

# Act on contract results.
def enforce(
    results: list[dict[str, str]],
) -> list[dict[str, str]]: ...
```

Starter code:

```python
class ContractViolation(Exception):
    ...


def enforce(results: list[dict[str, str]]) -> list[dict[str, str]]:
    ...
```

Exercise 5. Spot the bug (level 5)

This is the reference `ContractValidationResult.pass_rate`. It fails with a type error the first time it runs. Tap the line that causes it, then fix the imports.

```python
from dataclasses import dataclass, field
from pyspark.sql.functions import col, sum


@dataclass
class ContractValidationResult:
    checks: list = field(default_factory=list)

    @property
    def pass_rate(self) -> float:
        passed = sum(1 for c in self.checks if c.status == "PASS")
        return passed / len(self.checks) if self.checks else 0
```

Write down the line number of the bug. Then write the fix.

Exercise 6. Multiple choice (level 5)

Your silver table is built with Lakeflow Spark Declarative Pipelines. Rows with a NULL `respondent_id` must never reach the table, but the update should keep going for valid rows. Which decorator fits?

- a) `@dp.expect("valid_id", "respondent_id IS NOT NULL")`
- b) `@dp.expect_or_drop("valid_id", "respondent_id IS NOT NULL")`
- c) `@dp.expect_or_fail("valid_id", "respondent_id IS NOT NULL")`
- d) No decorator: filter in the next notebook

Why?

- r1) expect_or_drop removes violating rows and records how many were dropped.
- r2) expect_or_drop fails the update when any row violates.

Sources

- Importing sum from pyspark.sql.functions shadows the built-in, so sum over a generator fails. (verified: Run on PySpark 4.0.4: PySparkTypeError [NOT_COLUMN_OR_STR] Argument `col` should be a Column or str, got generator.)
- Lakeflow Spark Declarative Pipelines provide @dp.expect, @dp.expect_or_drop and @dp.expect_or_fail in pyspark.pipelines. (verified: https://docs.databricks.com/aws/en/ldp/developer/ldp-python-ref-expectations (checked 2026-09-25))

A6-L2 Statistical checks that survive empty, single-row and zero data

Concept

Statistical QA runs on every wave, including the strange ones: an empty delivery, a subgroup with one respondent, a metric that drops to exactly zero. A check that crashes on those cases fails exactly when you need it.

Know what Spark 4.0 returns at the edges (all tested on PySpark 4.0.4):

Case	Result
`F.mean`, `F.stddev`, `percentile_approx` on an empty DataFrame	NULL
`F.stddev` on one row	NULL (it is the sample standard deviation)

Case	Result
`F.col("a") / F.col("b")` where b is 0, ANSI mode on	Error: `DIVIDE_BY_ZERO`, even for DOUBLE
`F.try_divide(a, b)` where b is 0	NULL
Python `round(None, 4)`	`TypeError`

So a weight-spread check built as `round(stats["stddev"] / stats["mean"], 4)` fails three ways: NULL mean on an empty wave, NULL stddev on one row, and `round(None)` in Python.

Four habits make checks robust:

1. **Compute the count with the statistics**, in the same `agg`, and decide early. Fewer than two rows: return a `SKIP` result that says why.
2. **Divide with** `F.try_divide` or guard with `F.when(denominator > 0, ...)`. Never a bare `/` on data you do not control.
3. **Test for None with** `is None` in Python. `if cv:` treats a real CV of 0.0 as missing, and `if current_pct and prior_pct:` skips a share that fell to 0.0 (reference defect 8).
4. **Return a result for every check**, including SKIP. A check that silently returns nothing looks exactly like a check that passed.

Why it matters

A new tracker's first wave has one respondent in a small region, because of a routing test. The weight-spread check computes the sample standard deviation for that region: NULL. Python then runs `round(None, 4)` and raises `TypeError`. The QA job fails, and because QA gates publication, the whole wave's reporting is delayed by a day while someone reads the stack trace.

With the count checked first, that region gets a SKIP result, "fewer than 2 respondents", and every other region is checked as normal.

Worked example: weak code

```
def check_weight_distribution(df, weight_col):
    stats = df.agg(
        F.mean(weight_col).alias("mean"),
        F.stddev(weight_col).alias("stddev"),
    ).collect()[0]
    cv = stats["stddev"] / stats["mean"] if stats["mean"] else None
    return {
        "mean": round(stats["mean"], 4),
        "cv": round(cv, 4) if cv else None,
        "cv_warning": cv > 1.5 if cv else False,
```

```
11      }
```

1. **Line 6.** stddev is NULL for one row, so None / mean raises TypeError.
2. **Line 8.** An empty frame gives mean None, and round(None) raises TypeError.
3. **Line 10.** Truthiness: a missing CV reports no warning, as if the check had passed.

Worked example: strong code

```
1  MIN_ROWS = 2
2  MAX_WEIGHT_CV = 1.5
3
4
5  def weight_spread(df: DataFrame, weight_col: str) -> dict[str, float |
     int | None]:
6      """Count, mean, stddev and CV of weights in one pass. CV is NULL when
     undefined."""
7      row = df.agg(
(1)        F.count(weight_col).alias("n"),
9          F.mean(weight_col).alias("mean"),
10         F.stddev(weight_col).alias("stddev"),
(2)        F.try_divide(F.stddev(weight_col),
     F.mean(weight_col)).alias("cv"),
12     ).collect()[0]
13     return row.asDict()
14
15
16 def check_weight_spread(stats: dict[str, float | int | None]) ->
     CheckResult:
17     """PASS, WARN, or SKIP with a reason. Never raises on edge cases."""
(3)    if stats["n"] < MIN_ROWS or stats["cv"] is None:
19         return CheckResult("weight_cv", "SKIP", "MEDIUM", f">= {MIN_ROWS}
     rows", f"n={stats['n']}")
(4)    status = "WARN" if stats["cv"] > MAX_WEIGHT_CV else "PASS"
21     return CheckResult("weight_cv", status, "MEDIUM", f"<=
     {MAX_WEIGHT_CV}", f"{stats['cv']:.4f}")
```

1. **Line 8.** The count travels with the statistics, in the same pass.
2. **Line 11.** CV computed in Spark. try_divide returns NULL for a zero or NULL mean instead of raising.
3. **Line 18.** Edge cases become an explicit SKIP with the reason, checked with `is None`.
4. **Line 20.** Only reached with a real number, so the comparison is safe.

The strong version returns a result for every wave and every subgroup, and says why when a statistic is undefined.

Recap card

- **F.stddev on one row, and mean on an empty frame** Both NULL. stddev is the sample standard deviation.
- **Division by zero on Spark 4.0 with ANSI on** Raises DIVIDE_BY_ZERO, even for DOUBLE. Use F.try_divide or F.when(b > 0, a / b).
- **What should a check return when a statistic is undefined?** An explicit SKIP result with the reason. Never nothing, and never a crash.
- **Why is `if cv:` wrong?** A real CV of 0.0 is falsy. Test missing values with `is None`.

Practice

Exercise 1. Predict the output (level 1)

A region has exactly one respondent. What does this print?

```
one = spark.createDataFrame([(1.2,)], "final_weight DOUBLE")
print(one.agg(F.stddev("final_weight")).collect()[0][0])
```

Exercise 2. Multiple choice (level 2)

On Spark 4.0 (Runtime 17.3 LTS), `F.col("weighted_reach") / F.col("weighted_base")` meets a row where `weighted_base` is `0.0`. What happens?

- a) The result is Infinity.
- b) The result is NULL.
- c) The query fails with DIVIDE_BY_ZERO.
- d) The result is 0.0.

Why?

- r1) ANSI mode is on by default in Spark 4.0, and it raises on division by zero, DOUBLE included.
- r2) Delta rejects infinite values.

Exercise 3. Fill the gap (level 3)

Compute the coefficient of variation in Spark so that a zero or NULL mean gives NULL instead of an error.

```
cv = F._____(1)(F.stddev("final_weight"), F._____(2)("final_weight"))
```

Exercise 4. Write the function (level 4)

Write `check_weight_spread(stats, max_cv=1.5)`. `stats` is a dict with `n` (int), `cv` (float or None). Return a tuple `(status, reason)`:

- `("SKIP", "fewer than 2 rows")` when `n < 2`.
- `("SKIP", "cv undefined")` when `cv` is None.
- `("WARN", ...)` when `cv > max_cv`, otherwise `("PASS", ...)`. A CV of `0.0` is a real value and passes.

Your code must pass these checks:

- n below 2 must return ("SKIP", "fewer than 2 rows").
- A None CV with enough rows must return ("SKIP", "cv undefined").
- A CV of 0.0 means all weights are equal. It is a real value and passes.
- Above max_cv must WARN. Use the max_cv argument, not a fixed number.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Classify the weight coefficient of variation.
def check_weight_spread(
    stats: dict[str, float | int | None],
    max_cv: float = 1.5,
) -> tuple[str, str]: ...
```

Starter code:

```
def check_weight_spread(stats: dict[str, float | int | None], max_cv:
    float = 1.5) -> tuple[str, str]:
    ...
```

Exercise 5. Spot the bug (level 5)

This is the reference wave-stability comparison. A share that dropped from 12% to 0% produced no result at all. Tap the line, then fix the function.

```
def compare_share(metric: str, current_pct: float | None, prior_pct:
    float | None, tolerance: float = 0.10) -> dict[str, object] | None:
    """Return a stability result, or None when a share is missing."""
    if current_pct and prior_pct:
        change = abs(current_pct - prior_pct)
        return {"metric": metric, "change_pp": round(change, 4),
    "status": "WARN" if change > tolerance else "PASS"}
    return None
```

Write down the line number of the bug. Then write the fix.

Sources

- On PySpark 4.0.4: stddev of one row and mean/stddev/percentile_approx of an empty frame are NULL; division by zero raises DIVIDE_BY_ZERO for DOUBLE under ANSI; try_divide returns NULL; Python round(None) raises TypeError. (verified: Run in this repository.)

A6-L3 Reconcile counts and totals between layers

Concept

Each layer transforms data, and each transformation can lose or duplicate rows. **Reconciliation** checks that what came out of one layer matches what went into the next, for every wave you load.

Compare quantities that must agree by design:

From	To	Must match
Bronze, wave W	Silver, wave W	Distinct `(respondent_id, wave_id)` keys in bronze = silver rows
Silver, wave W, in scope	Gold, wave W	Sum of `final_weight` = sum of `weighted_base` over all subgroups
Silver, wave W, in scope	Gold, wave W	Row count = sum of `unweighted_n`

Pick the right bronze figure. Bronze holds every delivery, redeliveries included, so its raw row count is larger than silver's by design. Compare **distinct keys**.

Compare totals with a **tolerance**. Floating-point sums depend on the order Spark adds values in, which varies with partitioning, so two correct totals can differ in the last digits. In Python, `0.1 + 0.2 == 0.3` is `False`. Use `math.isclose(a, b, rel_tol=1e-9, abs_tol=1e-6)`. The `abs_tol` handles totals near zero, where a relative tolerance means nothing.

Compute each side with one aggregation, record the result as a structured check (expected, actual, status), and treat a failed reconciliation as **CRITICAL**. If gold does not add up to silver, every published figure for that wave is suspect.

Run reconciliation after each layer loads and before anything downstream reads it. Store the results with the run log, so you can show that every published wave reconciled.

Why it matters

A gold job joins silver to a region lookup with an inner join. A supplier introduces a new region code, `NI`, that the lookup does not have. 312 respondents drop out of gold without an error. Weighted reach for the wave looks plausible, because the base shrank along with the reach.

A reconciliation of silver's in-scope weight total against the sum of gold's `weighted_base` catches the 1.8% gap on the same run. The alert names the wave and the size of the difference, and the fix is a lookup update, not a client complaint.

Worked example: weak code

```
def reconcile(bronze, silver, wave):
    b = bronze.filter(F.col("wave_id") == wave).count()
    s = silver.filter(F.col("wave_id") == wave).count()
    if b != s:
        print(f"mismatch {b} vs {s}")
```

1. **Line 2.** Counts every delivery, redeliveries included. A redelivered wave always mismatches.
2. **Line 4.** Exact equality: fine for counts, wrong for weight totals.
3. **Line 5.** A print that nobody reads, and the pipeline carries on.

Worked example: strong code

```
import math


def silver_totals(silver: DataFrame, wave_id: str) -> tuple[int, float]:
    """In-scope row count and weight total for one wave, in one pass."""
    row = (
        silver.filter((F.col("wave_id") == wave_id) & F.col("in_scope"))
        .agg(F.count("*").alias("n"), F.coalesce(F.sum("final_weight"),
    F.lit(0.0)).alias("w"))
        .collect()[0]
    )
    return int(row["n"]), float(row["w"])


def gold_totals(gold: DataFrame, wave_id: str) -> tuple[int, float]:
    """Sum of unweighted_n and weighted_base over every subgroup of one
    wave."""
    row = (
        gold.filter(F.col("wave_id") == wave_id)
        .agg(F.coalesce(F.sum("unweighted_n"), F.lit(0)).alias("n"),
    F.coalesce(F.sum("weighted_base"), F.lit(0.0)).alias("w"))
        .collect()[0]
    )
```

```
21      return int(row["n"]), float(row["w"])
22
23
24  def totals_match(expected: float, actual: float) -> bool:
25      """Equal within floating-point tolerance, including totals near
        zero."""
(3)     return math.isclose(expected, actual, rel_tol=1e-9, abs_tol=1e-6)
```

1. **Line 7.** Same population as gold: one wave, in scope only.
2. **Line 8.** Count and total in one pass. Coalesce so an empty wave gives 0, not NULL.
3. **Line 26.** Tolerant comparison for floating-point totals.

Silver and gold totals are computed the same way and compared with tolerance. A mismatch becomes a CRITICAL result with numbers, not a print.

Recap card

- **Bronze figure that must match silver's row count for a wave** Distinct (respondent_id, wave_id) keys, not raw bronze rows, because bronze keeps every delivery.
- **Silver-to-gold reconciliation** In-scope silver weight total = sum of gold weighted_base; in-scope silver rows = sum of unweighted_n.
- **How to compare floating-point totals** math.isclose(a, b, rel_tol=1e-9, abs_tol=1e-6). Never ==.
- **Severity of a failed reconciliation** CRITICAL. Every figure for the wave is suspect until it passes.

Practice

Exercise 1. Multiple choice (level 1)

Wave W14 was delivered twice to bronze. What should the silver row count for W14 equal?

- a) The bronze row count for W14
- b) The number of distinct (respondent_id, wave_id) keys in bronze for W14
- c) Half the bronze row count for W14
- d) The row count of the latest delivery only

Why?

- r1) Silver keeps one row per key, the newest version, so it matches the number of distinct keys across all deliveries.
- r2) Bronze deletes old deliveries after silver loads.

Exercise 2. Predict the output (level 2)

What does this print?

```
import math

print(0.1 + 0.2 == 0.3, math.isclose(0.1 + 0.2, 0.3))
```

Exercise 3. Write the function (level 3)

Write `reconciliation_result(name, expected, actual)` that returns a dict with `check_name`, `status` (`"PASS"` or `"FAIL"`), `severity` (always `"CRITICAL"`), `expected` and `actual` (both formatted to 2 decimal places as strings), and `difference` (actual minus expected, a float). Compare with `math.isclose(expected, actual, rel_tol=1e-9, abs_tol=1e-6)`.

Your code must pass these checks:

- 0.3 and 0.1 + 0.2 must PASS. Use math.isclose.
- A 900,000 gap must FAIL, with difference = actual - expected.
- Totals near zero need abs_tol, or 0.0 against 1e-9 fails.
- Format expected and actual to two decimal places, and keep the check name.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Compare two layer totals and return a CRITICAL check result.
def reconciliation_result(
    name: str,
    expected: float,
    actual: float,
) -> dict[str, object]: ...
```

Starter code:

```
import math


def reconciliation_result(name: str, expected: float, actual: float) -> dict[str, object]:
    ...
```

Exercise 4. Spot the bug (level 4)

This bronze-to-silver reconciliation fails every time a wave is redelivered, even when silver is correct. Tap the line, then fix the function.

```
def bronze_silver_keys(bronze: DataFrame, silver: DataFrame, wave_id: str) -> tuple[int, int]:
    """Keys in bronze and rows in silver for one wave."""
    b = bronze.filter(F.col("wave_id") == wave_id).count()
```

```
    s = silver.filter(F.col("wave_id") == wave_id).count()
    return b, s
```

Write down the line number of the bug. Then write the fix.

Exercise 5. Put in order (level 5)

Order the steps for one wave, from load to publication.

- MERGE the wave into silver
- Store reconciliation results with the run log
- Reconcile silver in-scope totals with gold totals
- Refresh the Power BI semantic model
- Build gold metrics for the wave
- Reconcile bronze distinct keys with silver rows

Practical: Reconcile silver with gold, and catch a dropped region

Platform: Databricks Free Edition. Run this in Databricks Free Edition. It builds a small silver table and two gold versions: one correct, one built with an inner join that drops an unknown region. The reconciliation passes for the first and fails for the second.

Download the notebook `A6-L3.ipynb` from the app.

Sources

- 0.1 + 0.2 == 0.3 is False and math.isclose(0.1 + 0.2, 0.3) is True in Python. (verified: Run in Python 3.11 in this repository.)
- F.count_distinct exists in PySpark 4.0. (verified: Run on PySpark 4.0.4 in this repository.)

Track A best-practice checklist

☐ Every function has type hints, a docstring and one job.

☐ No SparkSession is created at import time. Functions receive spark as a parameter.

☐ Functions are imported as F: `from pyspark.sql import functions as F`.

☐ Table names, column maps and thresholds come from config or registry tables.

☐ Every table states its grain in a comment or table property.

☐ Bronze appends. Silver deduplicates the source, then MERGEs. Gold aggregates by every grain column.

☐ Every run writes one record to the run log table with an explicit schema.

☐ CRITICAL checks raise. WARN checks log and continue.

☐ Every division has a guard for zero and null.

☐ Counts come from one agg pass, not repeated count() calls.

☐ Incremental loads compare source and target on the same keys.

☐ Row filters use a SQL UDF with SET ROW FILTER. Grants go to groups.

☐ Tests run on a local SparkSession in CI before every merge.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

A6-L1 Data contracts with severity levels and structured results

A6-L1 exercise 1

Hints. (1) Would the problem corrupt totals or the grain? (2) Or is it something a person should look at, with no immediate harm? (3) Keys and weights are CRITICAL. Labels and small shifts are WARN.

Answer.

- Duplicate keys break the silver grain and the MERGE: CRITICAL.
- A weight total out of range makes every weighted figure wrong: CRITICAL.
- A few unexpected labels are worth investigating but do not corrupt totals: WARN.
- A small distribution shift can be real change: WARN, reviewed by a person.

CRITICAL is for problems that make published numbers wrong or break the table. WARN is for anomalies that need a human look.

Common wrong answers.

- keys = `warn`: Duplicate keys fail the MERGE or break the grain. Silver must not receive them.
- weights = `warn`: Every weighted estimate depends on the weight total. Publishing with wrong weights is worse than publishing late.
- category = `critical`: Twelve odd labels should not stop the whole wave. Log them and follow up with the supplier.

A6-L1 exercise 2

Hints. (1) How many round trips with the supplier do you want? (2) Which failures must stop the run? (3) Store all results, then raise once.

Answer. b) All three results stored, then one exception naming both CRITICAL failures. Because: r1) Evaluate everything, record everything, then act on severity once.

Run every check, store every result, then raise one exception listing all CRITICAL failures. The WARN is stored and reviewed.

Common wrong answers.

- `a`: Stopping at the first failure hides the second, and the supplier needs another round trip.
- `c`: Printed text is lost. Results must be records in a table.
- `d`: Two CRITICAL failures must stop the run before silver.
- `b|r2`: Right outcome, wrong reason. Here you deliberately raise late, after all checks have run, so one run reports everything.

A6-L1 exercise 3

Hints. (1) The typing construct for exact values. (2) The dataclass option that forbids changing fields after creation. (3) `Literal` and `frozen`.

Answer.

```
Status = Literal["PASS", "WARN", "FAIL", "SKIP"]


@dataclass(frozen=True)
class CheckResult:
    check_name: str
    status: Status
```

`Literal[...]` lists the allowed strings. `frozen=True` means no code can change a result after the check records it.

Common wrong answers.

- g1 = `Union`: `Union` combines types, not values. A fixed set of strings is a `Literal`.
- g2 = `slots`: `slots=True` saves memory but still allows changes. `frozen=True` makes a result immutable once recorded.

A6-L1 exercise 4

Hints. (1) Collect all CRITICAL failures into a list first. (2) Raise once, joining their names into the message. (3) Return the WARN results when nothing CRITICAL failed.

Answer.

```
class ContractViolation(Exception):
    """Raised when one or more CRITICAL checks fail."""


def enforce(results: list[dict[str, str]]) -> list[dict[str, str]]:
    """Act on contract results.

    Args:
        results: Every check result for one delivery.
```

```
10
11      Returns:
12          The WARN results, for logging.
13
14      Raises:
15          ContractViolation: If any CRITICAL check failed. The message names
    each one.
16      """
17      critical = [r for r in results if r["status"] == "FAIL" and
    r["severity"] == "CRITICAL"]
18      if critical:
19          names = ", ".join(f"{r['check_name']} (expected {r['expected']},
    got {r['actual']})" for r in critical)
20          raise ContractViolation(f"{len(critical)} critical check(s)
    failed: {names}")
21      return [r for r in results if r["status"] == "WARN"]
```

Collecting before raising means one run reports every CRITICAL failure. Filtering on both status and severity keeps a MEDIUM failure from stopping the pipeline.

A6-L1 exercise 5

Hints. (1) Which `sum` does line 11 call? (2) An import can replace a built-in name for the whole module. (3) Import `functions as F` instead of importing `sum` by name.

Answer. The bug is on line 2.

Fixed version:

```
1  from dataclasses import dataclass, field
2  from pyspark.sql import functions as F
3
4
5  @dataclass
6  class ContractValidationResult:
7      checks: list = field(default_factory=list)
8
9      @property
10     def pass_rate(self) -> float:
11         passed = sum(1 for c in self.checks if c.status == "PASS")
12         return passed / len(self.checks) if self.checks else 0
```

Line 2 imports Spark's `sum`, which replaces Python's built-in for the whole module. Line 11 then passes a generator to a Spark column function, which fails. Import the module as `F` and use `F.sum` where you need Spark's version. This is defect 2 in your reference README.

Common wrong answers.

- Line 11: The generator and the built-in sum are correct. But which `sum` does this line call?
- Line 12: The empty-list guard is fine. The failure happens one line earlier, caused by an import.

A6-L1 exercise 6

Hints. (1) Three behaviours: keep, drop, fail. (2) The requirement says never reach the table, but keep going. (3) Drop.

Answer. b) `@dp.expect_or_drop("valid_id", "respondent_id IS NOT NULL")` Because: r1) expect_or_drop removes violating rows and records how many were dropped.

`expect` keeps and records, `expect_or_drop` drops and records, `expect_or_fail` stops. Choose the one that matches the severity of the rule.

Common wrong answers.

- `a`: `expect` keeps violating rows and only records metrics. NULL keys would reach the table.
- `c`: `expect_or_fail` stops the whole update on the first violation. The requirement keeps valid rows flowing.
- `d`: A later filter lets bad rows land first, and loses the violation metrics.
- `b|r2`: Right decorator, wrong reason. Failing the update is expect_or_fail. expect_or_drop drops the rows and keeps going.

A6-L2 Statistical checks that survive empty, single-row and zero data

A6-L2 exercise 1

Hints. (1) `F.stddev` is an alias for `stddev_samp`. (2) The sample formula divides by n - 1. (3) With n = 1 the result is NULL, printed as None.

Answer.

```
1 None
```

`F.stddev` computes the sample standard deviation, dividing by n - 1. For one row that is undefined, so the result is NULL. Your checks must expect it.

Common wrong answers.

- `0.0`: `F.stddev` is the sample standard deviation, which divides by n - 1. With one row that is undefined, so Spark returns NULL.

- `1.2`: That is the mean. The spread of a single value is undefined for the sample formula.

A6-L2 exercise 2

Hints. (1) Which mode is on by default in Spark 4.0? (2) Does that mode tolerate division by zero? (3) It raises, and the message suggests try_divide.

Answer. c) The query fails with DIVIDE_BY_ZERO. Because: r1) ANSI mode is on by default in Spark 4.0, and it raises on division by zero, DOUBLE included.

Under ANSI mode, division by zero raises `DIVIDE_BY_ZERO`, even for doubles. Use `F.try_divide(a, b)` or `F.when(b > 0, a / b)`.

Common wrong answers.

- `a`: Without ANSI mode some engines return Infinity for doubles. Spark 4.0 with ANSI on raises DIVIDE_BY_ZERO.
- `b`: NULL is what `F.try_divide` returns. The plain `/` raises under ANSI mode.
- `d`: No engine returns 0.0 for a division by zero. Spark 4.0 raises.
- `c|r2`: Right outcome, wrong reason. The error comes from the ANSI division, before anything is written.

A6-L2 exercise 3

Hints. (1) The safe division function starts with try. (2) CV = standard deviation / mean. (3) `try_divide` and `mean`.

Answer.

```
cv = F.try_divide(F.stddev("final_weight"), F.mean("final_weight"))
```

`F.try_divide(F.stddev(w), F.mean(w))` gives the CV, or NULL when the mean is 0 or NULL.

Common wrong answers.

- g1 = `divide`: There is no `F.divide`. The safe version is `F.try_divide`, which returns NULL for a zero divisor.
- g2 = `stddev`: The CV divides the standard deviation by the mean.

A6-L2 exercise 4

Hints. (1) Check n first, then cv with `is None`. (2) Only compare cv with max_cv once you know it is a number. (3) Return the status and a short reason in a tuple.

Answer.

```python
def check_weight_spread(stats: dict[str, float | int | None], max_cv: float = 1.5) -> tuple[str, str]:
    """Classify the weight coefficient of variation.

    Args:
        stats: n (row count) and cv (None when undefined).
        max_cv: Largest CV that passes.

    Returns:
        (status, reason), where status is SKIP, WARN or PASS.
    """
    n = stats["n"]
    cv = stats["cv"]
    if n is None or n < 2:
        return ("SKIP", "fewer than 2 rows")
    if cv is None:
        return ("SKIP", "cv undefined")
    if cv > max_cv:
        return ("WARN", f"cv {cv:.4f} above {max_cv}")
    return ("PASS", f"cv {cv:.4f}")
```

Ordering the checks means the comparison only ever sees a real number. `is None` keeps a CV of 0.0 as a pass.

A6-L2 exercise 5

Hints. (1) What is `0.0 and 0.12` in Python? (2) The block is skipped for a share of exactly zero. (3) Use `is not None` on both shares.

Answer. The bug is on line 3.

Fixed version:

```python
def compare_share(metric: str, current_pct: float | None, prior_pct: float | None, tolerance: float = 0.10) -> dict[str, object] | None:
    """Return a stability result, or None when a share is missing."""
    if current_pct is not None and prior_pct is not None:
        change = abs(current_pct - prior_pct)
        return {"metric": metric, "change_pp": round(change, 4), "status": "WARN" if change > tolerance else "PASS"}
    return None
```

`0.0` is falsy, so the largest possible drop skipped the comparison. This is defect 8 in your reference README. Test for missing values with `is not None`.

Common wrong answers.

- Line 5: The rounding and status logic are fine once reached. The share of 0.0 never reaches them.
- Line 4: The subtraction is correct. The problem is the condition guarding it.

A6-L3 Reconcile counts and totals between layers

A6-L3 exercise 1

Hints. (1) What does silver keep when a key appears twice? (2) What does bronze keep? (3) Count distinct keys in bronze.

Answer. b) The number of distinct (respondent_id, wave_id) keys in bronze for W14 Because: r1) Silver keeps one row per key, the newest version, so it matches the number of distinct keys across all deliveries.

Silver holds one row per `(respondent_id, wave_id)`. The matching bronze figure is the number of distinct keys across all deliveries of the wave.

Common wrong answers.

- `a`: Bronze holds both deliveries, so its row count is roughly double. Silver keeps one row per key.
- `c`: Only if both deliveries hold exactly the same keys. Distinct keys is correct in every case.
- `d`: A redelivery can add or drop respondents. Silver keeps every key ever delivered for the wave, at its newest version.
- `b|r2`: Right figure, wrong reason. Bronze is append-only and keeps every delivery.

A6-L3 exercise 2

Hints. (1) Can 0.1 be stored exactly as a binary float? (2) What is 0.1 + 0.2 printed in full? (3) isclose allows a tiny relative difference.

Answer.

```
1 False True
```

`0.1 + 0.2` is `0.30000000000000004`, so `==` is False. `math.isclose` allows a relative difference of 1e-9, so it is True. Weight totals need the same treatment.

Common wrong answers.

- `True True`: 0.1 and 0.2 have no exact binary representation, so their sum is 0.30000000000000004. Exact equality fails.
- `False False`: `math.isclose` compares with a relative tolerance of 1e-9 by default, so it treats the two values as equal.

A6-L3 exercise 3

Hints. (1) `math.isclose(expected, actual, rel_tol=1e-9, abs_tol=1e-6)`. (2) Format numbers with an f-string: `f"{x:.2f}"`. (3) `difference` is `actual - expected`, as a float.

Answer.

```
import math


def reconciliation_result(name: str, expected: float, actual: float) -> dict[str, object]:
    """Compare two layer totals and return a CRITICAL check result.

    Args:
        name: Check name, for example weight_total_silver_gold.
        expected: Total from the upstream layer.
        actual: Total from the downstream layer.

    Returns:
        A check result with status PASS or FAIL and the difference.
    """
    ok = math.isclose(expected, actual, rel_tol=1e-9, abs_tol=1e-6)
    return {
        "check_name": name,
        "status": "PASS" if ok else "FAIL",
        "severity": "CRITICAL",
        "expected": f"{expected:.2f}",
        "actual": f"{actual:.2f}",
        "difference": actual - expected,
    }
```

`rel_tol` absorbs rounding in large totals. `abs_tol` covers totals near zero, where any relative tolerance is meaningless.

A6-L3 exercise 4

Hints. (1) What does bronze hold after a redelivery? (2) Silver has one row per key. Which bronze figure matches that? (3) Count distinct respondent IDs within the wave.

Answer. The bug is on line 3.

Fixed version:

```
def bronze_silver_keys(bronze: DataFrame, silver: DataFrame, wave_id: str) -> tuple[int, int]:
    """Keys in bronze and rows in silver for one wave."""
    b = bronze.filter(F.col("wave_id") == wave_id).agg(F.count_distinct("respondent_id")).collect()[0][0]
    s = silver.filter(F.col("wave_id") == wave_id).count()
    return b, s
```

Bronze keeps every delivery, so its raw count grows with each redelivery. Count distinct keys for the wave. Within one wave, distinct `respondent_id` is the same as distinct `(respondent_id, wave_id)`.

Common wrong answers.

- Line 4: Silver has one row per key, so its row count is the right figure. The bronze side is wrong.

A6-L3 exercise 5

Hints. (1) Each layer is checked before the next reads it. (2) Nothing is published before every check passes. (3) Record the evidence, then publish.

Answer.

1. MERGE the wave into silver
2. Reconcile bronze distinct keys with silver rows
3. Build gold metrics for the wave
4. Reconcile silver in-scope totals with gold totals
5. Store reconciliation results with the run log
6. Refresh the Power BI semantic model

Load, check, load, check, record, publish. A failed reconciliation stops the sequence before clients see anything.

Common wrong answers.

- Reconcile silver before building gold, so a bad silver load never feeds gold.
- Store the results before publishing, so the published wave has a record of passing.
- Publishing before the silver-to-gold reconciliation lets unreconciled numbers reach clients.